

FINAL YEAR PROJECT

Sentinel AI Implementation Report

A clear summary of what has been built through Task 5, where every part is located, what has been tested, and what is still pending.

Current status: Task 5 validation work completed; M4 approval pending

Project: Adaptive Financial Fraud Detection

Branch reviewed: ritwik

Report date: 14 August 2026

Coverage: Repository foundation, frontend, backend/server, PostgreSQL, Redis streaming, offline machine learning, Task 5 ensemble, infrastructure and tests

1. Project summary

Sentinel AI is being developed as a financial-fraud detection system. The implemented project can accept safe synthetic or tokenized authorization events, store audit information, reliably send accepted events through Redis Streams, and run offline fraud-model experiments using the ULB credit-card dataset.

Task 5 adds calibrated probabilities, a hybrid ensemble and measured Allow/Review/Block threshold profiles. This work currently uses training and validation data only. The final test partition remains locked until the required approvals are given.

2. Where each part is implemented

Area	Purpose	Main location	Current state
Frontend	The page seen in a browser	<code>frontend/</code>	Basic React web shell only
Backend / API	Receives requests, validates events and returns responses	<code>backend/src/</code>	Working Express API
Server entry	Starts the backend HTTP process	<code>backend/src/server.js</code>	Working
Background server workers	Move and consume events through Redis Streams	<code>backend/src/stream/</code>	Publisher and consumer working
Database	Stores events, attempts, idempotency, outbox and processing receipts	<code>backend/migrations/</code>	PostgreSQL schema implemented
Machine learning	Dataset audit, baseline models, calibration, ensemble and policies	<code>services/ml/</code>	Tasks 4 and 5 validation work implemented
Local infrastructure	Starts all components together	<code>infrastructure/compose.yaml</code>	Docker Compose stack working
Automation and tests	Installs, checks and verifies the project	<code>scripts/</code> and component test folders	Automated verification implemented

Simple meaning: “Backend” is the complete server-side area. `server.js` starts its web server, while the publisher and consumer are extra backend processes that run in the background. The Python ML code is kept separately so that model experiments do not get mixed with the JavaScript API.

3. Work completed by task

Task 1 — Repository foundation

- Created a clean structure for frontend, backend, infrastructure, data, scripts, tests and documentation.
- Added local environment placeholders in `.env.example`.
- Added Git safety rules so secrets, private task PDFs, raw datasets, Kaggle credentials, caches and model artifacts are not committed.
- Added project instructions and basic documentation.

Main locations: repository root, `.gitignore`, `.env.example`, `AGENTS.md` and the component README files.

Task 2 — Local development and verification

- Added repeatable bootstrap, verification and cleanup commands for Windows PowerShell and Bash.
- Added Docker health checks, localhost-only ports and isolated PostgreSQL/Redis volumes.
- Created safe commands to start, inspect and stop the project without deleting database volumes.

Main locations: `scripts/`, `Makefile` and `infrastructure/compose.yaml`.

Task 3 — Authorization-event application boundary

- Replaced the old server shell with a JavaScript Node.js 22 and Express API.
- Implemented `GET /health`, `GET /ready` and `POST /api/v1/authorization-events`.
- Added validation, correlation IDs, idempotency protection, accepted/quarantined/rejected outcomes and structured logs.
- Added PostgreSQL audit persistence and transactional rollback behavior.
- Added an OpenAPI contract at `/openapi.json` and local Swagger documentation at `/docs`.
- Created a JavaScript React/Vite frontend shell and its test/build setup. No TypeScript is used.

Backend locations: `backend/src/app.js`, `controllers/`, `routes/`, `services/`, `repositories/` and `contracts/`.

Frontend locations: `frontend/src/App.jsx`, `main.jsx` and `styles.css`.

Task 4 Lane A — Reliable event streaming

- Added a PostgreSQL transactional outbox so an accepted event and its publication record are saved together.
- Added a Redis Streams publisher that reads outbox rows, validates the envelope and publishes safely.
- Added a consumer group with acknowledgements, retry handling, recovery of idle messages and dead-letter handling.
- Added deduplication receipts so repeated message delivery does not create repeated effects.
- Added evidence and integration checks for delivery, retry, recovery and failure behavior.

Main locations: `backend/src/stream/`, `backend/src/repositories/streamRepository.js`, `backend/migrations/0002_reliable-event-streaming.sql` and the `publisher/consumer` Compose services.

Task 4 Lane B — Offline machine-learning baselines

- Added KaggleHub loading for `mlg-ulb/creditcardfraud` and local CSV loading.
- Validated exactly 284,807 rows, 31 columns, 284,315 legitimate transactions and 492 fraud transactions.
- Added dataset schema, null, finite-number, duplicate, range and SHA-256 audit evidence.
- Created a chronological train/validation/test split.
- Trained Logistic Regression, Linear SVM, Random Forest and KNN baselines.
- Kept the Task 4 test partition unscored and kept CSV/model artifacts out of Git.

Main locations: `services/ml/src/fraud_ml/data.py`, `split.py`, `models.py`, `evaluation.py` and `cli.py`.

Task 5 — Calibrated hybrid ensemble and decision profiles

- Added deterministic five-fold training-only out-of-fold scoring for all four models.
- Compared sigmoid and isotonic calibration; isotonic was selected for all four models.
- Converted every model output into bounded probabilities. Raw SVM margins are never directly mixed into the ensemble.
- Compared equal-weight and constrained weighted soft voting. Weighted soft voting was selected.
- Added fusion eligibility checks, single-model comparison, leave-one-model-out analysis, score correlations and disagreement evidence.
- Created Conservative Block, Balanced Demo and Recall First validation profiles.
- Added model artifact checksums, corruption detection and safe loading rules.
- Repeated the complete development run. Labels, model probabilities and ensemble probabilities matched exactly.

Main locations: `services/ml/src/fraud_ml/task5/`, `services/ml/tests/task5/` and `services/ml/reports/task5/`.

4. Task 5 measured validation result

The selected weighted ensemble uses all four models:

Model	Selected calibration	Fusion weight
Logistic Regression	Isotonic	24.72%
Random Forest	Isotonic	25.62%
Linear SVM	Isotonic	24.55%
KNN	Isotonic	25.11%

The validation partition contains 56,957 rows, including 57 fraud cases.

Profile	Fraud caught	False alerts	Total action rate	Meaning
Conservative Block	46 of 57 (80.7%)	97	0.25%	Very selective
Balanced Demo	46 of 57 (80.7%)	97	0.25%	Recommended profile
Recall First	49 of 57 (86.0%)	531	1.02%	Catches three more frauds but creates many more alerts

These are validation results on a historical anonymized dataset. They are not proof of live-bank performance, money saved, production latency, fairness or customer impact. The final 56,962-row test partition has not been evaluated.

5. Testing and verification completed

- Backend formatting, linting and JavaScript unit tests passed.
- Real PostgreSQL migration, concurrency, rollback and integration tests passed.
- Redis streaming delivery, retry, recovery, deduplication and dead-letter checks passed.
- Frontend formatting, linting, component test and production build passed.
- The latest Python ML suite passed all 10 tests.
- Task 5 generated 11 private artifact files and all checksums were verified.
- The Task 5 full-data run was repeated; prediction outputs were exactly reproducible.

Verification statements above describe checks that were actually run in the local project workspace.

6. Local application layout

Running process	Technology	Local access / role
Web	React + Vite	http://127.0.0.1:15173
API server	Node.js + Express	http://127.0.0.1:18000
Publisher worker	Node.js	PostgreSQL outbox to Redis Stream
Consumer worker	Node.js	Consumes Redis messages and records proof receipts
PostgreSQL	PostgreSQL 16	127.0.0.1:15432
Redis	Redis 7	127.0.0.1:16379
Offline ML commands	Python 3.12, pandas and scikit-learn	Run separately from <code>services/ml/</code>

7. Privacy and Git protection

The repository ignores the following private or generated items:

- Real `.env` files, keys, credentials and Kaggle authentication files.
- The raw `creditcard.csv` dataset and the complete ML data directory.
- Trained model files, experiment artifacts, caches, logs and build output.
- Private task instruction PDFs and internal planning material under `docs/planning/`.

Only safe example configuration, source code, tests and aggregate Task 5 reports are intended for Git.

8. Current limitations and next approval

- The frontend is currently an operational shell. A working fraud dashboard, analyst screen and live transaction display have not been built.
- The ML ensemble is offline and is not connected to the Express API or Redis consumer.
- No production authentication, cloud deployment, monitoring, drift detection or automatic retraining has been implemented.
- No real financial or cardholder data should be sent to this project.
- Task 5 is paused at M4. The owner must select a threshold profile before the model bundle can be frozen.
- After freezing, separate M5 and M6 approval is required before exactly one final test evaluation.

Recommended next action: approve the **Balanced Demo** profile for M4. Then create and verify the frozen bundle. Do not evaluate the locked test partition until separate M6 permission is provided.